CSE 438 — Digital Image Processing

Assignment Part B: Self-Supervised Learning for Semantic Segmentation

Fine-tuning Pretrained SSL Encoders — SimCLR, BYOL, MAE, DINOv2

Dr. Md Rifat Ahmmad Rashid
Associate Professor, Dept. of CSE
rifat.rashid@ewubd.edu
Summer 2026

Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

Mode:	Group (same as Part A)	**Deadline (Sec. 3):**	**17.08.2026, 11:59 PM**
Prerequisite:	Part A submitted & evaluated	**Deadline (Sec. 4):**	**18.08.2026, 11:59 PM**
Evaluation:	Kaggle Notebooks (public links) + LaTeX report, submitted via Google Classroom, followed by Viva Voce		
SSL Methods:	• SimCLR (contrastive)	• BYOL (negative-free)	
	• MAE (masked autoencoder)	• DINOv2 (self-distillation)	

1. Overview

Note — This is Part B of a Two-Part Assignment

Part A asked each group to benchmark three **fully-supervised** segmentation architectures and identify a **best-performing decoder/head design** for your assigned dataset. **Part B** reuses that same dataset and that same best decoder design, but replaces full-supervision with **self-supervised pretraining**: you will pretrain (or load a pretrained) SSL encoder on **unlabelled** images, then attach your Part-A decoder and fine-tune the combined model using only a **small labelled subset**. The goal is to answer a central question in modern computer vision: *how much labelled data do you really need if the encoder has already learned good visual representations?*

1.1. The SSL-for-Segmentation Pipeline

Every one of the four methods in this assignment follows the same five-stage pipeline. Understand this pipeline before writing any code — you will be asked to explain each arrow at viva.

What Changes vs. Part A

- In Part A, the encoder (ResNet / MiT / CSP backbone) was trained **jointly and supervised** together with its decoder, on labelled data from epoch 1.
- In Part B, the encoder is **pretrained first**, with **no labels at all**, using a self-supervised pretext task (contrastive matching, masked-patch reconstruction, or self-distillation — never class labels).
- Only **after** pretraining does the encoder ever see a class label, and even then it sees far fewer labelled images than Part A used.

1.2. Learning Objectives

By the end of Part B, each student should be able to:

- Explain the pretext task behind SimCLR, BYOL, MAE, and DINOv2, and why none of them requires class labels.
- Map an existing leakage-safe train/validation/test split onto self-supervised roles — unlabelled

pretraining data, labelled fine-tuning data, and downstream validation/evaluation data — without re-shuffling or leaking images across roles.

- Pretrain (or load and adapt a pretrained) SSL encoder and extract dense patch-level features suitable for per-pixel classification.
- Attach a decoder head — reusing the best-performing design identified in Part A — on top of a frozen or fine-tuned SSL encoder.
- Fine-tune the combined encoder–decoder model using a small fraction of labelled data and evaluate it with standard segmentation metrics.
- Compare label-efficiency: how close does each SSL method get to the Part-A fully supervised result while using far less labelled data?
- Defend every design and hyperparameter choice at a live viva examination.

2. Data Split — Reusing Part A's Train / Validation / Test Roles

Critical — Reuse the Part-A Split, Do Not Re-shuffle

Start from the **identical** leakage-safe train/validation/test split you produced and saved in Part A (Notebook 0). **Do not** re-split, re-shuffle, or carve any new sub-pools out of it — this guarantees the Part A vs. Part B comparison is fair and keeps the exercise simple. Part B simply assigns each of the three existing Part-A splits a **new role**:

Part-A Split	New Role in Part B	Notes
Train	Unlabelled SSL pretraining corpus (Task B)	**Every** image in the Part-A training split is used, images only — masks are ignored/discarded during pretraining.
Validation	Labelled fine-tuning set (Task D)	**Every** image in the Part-A validation split is used *with* its mask as the (small) labelled training data for supervised fine-tuning.
Test	Downstream validation *and* final evaluation (Tasks D & E)	The Part-A test split is used **twice**: to monitor fine-tuning progress and select the best checkpoint (Task D), and afterwards to report final test-set metrics (Task E). See the caveat box below.

Report the exact image counts inherited from Part A for each split (train, validation, test) in your data-prep notebook's markdown cell, and reuse this identical role mapping for **all four** SSL methods so the comparison is apples-to-apples.

Caveat — The Test Split Now Serves a Double Role

Using the same test split both to monitor/checkpoint-select during fine-tuning **and** to report final metrics is a deliberate simplification for this assignment — with an already small labelled (validation) set, holding out yet another slice would leave too little data to fine-tune on. It is, however, a genuine departure from strict practice (normally a test set is touched exactly once, at the very end). You must:

- State this explicitly in your report (Section 6.3) rather than presenting the numbers as if the test set were untouched during training.
- Understand and be able to explain at viva **why** this inflates reported metrics somewhat relative

to a fully independent test set, and what the "correct" fix would be (e.g. carving out a fourth, truly-held-out slice if more labelled data were available).

3. Required Tasks

3.1. Task A — Split Role Mapping & Loader Preparation

- Load the Part-A train/validation/test file lists directly (no re-splitting); save a copy of the index/file lists as artifacts in this notebook for the other notebooks to reuse.
- For the **train split**, prepare a data loader that yields **augmented image views only** (no masks) — the exact augmentation pipeline differs per SSL method (Task B specifies this per-method).
- For the **validation split**, reuse your Part-A augmentation pipeline (synchronized image+mask transforms) to build the labelled fine-tuning loader (Task D).
- For the **test split**, build a loader with **no** augmentation (deterministic resize/normalize only) — it will be used, unaugmented, for both downstream checkpoint-selection monitoring and final evaluation (Tasks D & E).
- Produce a sanity-check grid showing: (a) two augmented views of the same train-split image side-by-side, and (b) a validation-split image with mask overlay — confirming both pipelines are correctly wired before any training begins.

3.2. Task B — Self-Supervised Pretraining (all four methods, 50 epochs)

Each group pretrains **all four** SSL encoders below on the **Part-A training split** (used here, images-only, as the unlabelled corpus) for **50 epochs** (a fixed requirement — see the note below on when 50 epochs of pretraining and 50 epochs of downstream fine-tuning may need to live in separate notebooks). Starter/example notebooks for each method will be shared separately on Google Classroom — you are expected to adapt them to your assigned dataset, not run them unmodified.

Method	Pretext Task	Typical Backbone	Key Idea
SimCLR	Contrastive matching	ResNet-50	Maximize agreement between two augmented views of the same image vs. other images (NT-Xent loss).
BYOL	Negative-free self-prediction	ResNet-50 (online + target)	Online network predicts target network's representation of a differently augmented view; no negative pairs needed.
MAE	Masked-patch reconstruction	ViT-Base/Large	Randomly mask ∼75% of image patches; encoder sees only visible patches, lightweight decoder reconstructs the masked pixels.
DINOv2	Self-distillation	ViT-S/14 or ViT-B/14	Student network matches teacher network's output distribution over multi-crop views; teacher is an EMA of the student.

Compute Budget — Pretraining From Scratch vs. Adapting a Checkpoint

Full SSL pretraining from random initialization can take days even on large GPUs. On Kaggle's free tier, you have two acceptable paths — state clearly in your report which path each method used:

1. **Load an official ImageNet-pretrained SSL checkpoint** (e.g. a released SimCLR / BYOL / MAE / DINOv2 checkpoint) and **continue self-supervised pretraining ("SSL fine-tuning")** on the Part-A train split for the required **50 epochs**. This is the **recommended default** for all four methods.
2. **Pretrain from scratch** on the Part-A train split for the required **50 epochs** if your group has extra compute and wants the extra challenge.

In both cases, report wall-clock time per epoch and plot the pretext-task loss curve (SimCLR/BYOL/DINOv2: contrastive/distillation loss; MAE: reconstruction loss).

Epoch Requirement & When to Split Into Two Notebooks

- **SSL pretraining (Task B): 50 epochs. Downstream fine-tuning (Task D): 50 epochs.** Both counts are fixed requirements for every method.
- By default, one notebook per SSL method covers **both** stages (pretraining → decoder attachment → fine-tuning), as in Section 4.
- **If either stage's per-epoch time exceeds 10 minutes** on your Kaggle GPU (i.e. 50 epochs would take >8–9 hours and risk hitting the session time limit), **split that method into two separate notebooks**:
  - a **pretraining notebook** that runs the 50-epoch SSL pretext task and saves the resulting encoder checkpoint as a Kaggle output/dataset, and
  - a **downstream notebook** that loads that saved encoder checkpoint, attaches the Part-A decoder, and runs the 50-epoch supervised fine-tuning (Task C–F).
- Report the measured per-epoch time for both stages in your config cell even when you do **not** split, so the choice is auditable at viva.
- You may split some SSL methods and not others — e.g. keep SimCLR/BYOL (ResNet, faster) in one notebook each, but split MAE/DINOv2 (ViT, typically slower per epoch) into pretrain + downstream notebook pairs. State your choice and the timing that justified it in the report (Section 6.5).

3.3. Task C — Attaching the Decoder Head

- Identify the decoder/head design your group found best-performing in Part A (e.g. the ASPP head from DeepLabV3, the All-MLP head from SegFormer, or the segmentation head from YOLOv26-sem) and re-implement it (or reuse the Part-A code) on top of each SSL encoder's dense patch-feature output.
- Because encoder patch/feature resolutions differ (CNN feature maps vs. ViT patch tokens), add whatever adapter is needed — e.g. reshaping ViT token sequences back to a 2-D grid before your decoder's upsampling stages. Document this adapter explicitly.
- State clearly, per SSL method, whether the encoder is kept **frozen** (linear probing of the decoder only) or **fully fine-tuned** during Task D — you must try **both** for at least one SSL method and compare, then choose one strategy for the remaining three.

3.4. Task D — Supervised Fine-tuning (Validation Split as Labelled Data, Downstream: 50 epochs)

- Fine-tune each encoder+decoder pair using **only** the Part-A validation split (your labelled fine-tuning data, per Section 2) for exactly **50 epochs** (this is a fixed requirement matching the 50-epoch SSL pretraining stage, not a floor — report the final downstream-validation mIoU at epoch 50 even if it is still trending upward).

- **Monitor** per-epoch loss and mIoU on the Part-A **test split** during fine-tuning and save/select the best checkpoint by this signal — per Section 2, the test split doubles as your downstream validation set in this assignment.
- Collect all hyperparameters in a single visible config cell per notebook: optimizer, learning rate (separate LR for encoder vs. decoder if used), schedule, batch size, input resolution, loss function, frozen-vs-fine-tuned flag, and total training time.
- Plot the fine-tuning loss curve (on the validation-split labelled data) alongside the monitoring loss/mIoU curve (on the test split) for every method.

3.5. Task E — Testing, Metrics & Label-Efficiency Comparison

Evaluate all four SSL→fine-tuned models on the **same test split** used for monitoring in Task D (and identical to Part A's test split), reporting:

- Mean IoU (mIoU) and per-class IoU, Pixel Accuracy, Dice coefficient, confusion matrix (identical metric set to Part A Task F).
- A **label-efficiency comparison table**: each SSL method's test mIoU (fine-tuned on only the Part-A **validation split**) side-by-side with your Part-A best fully-supervised model's test mIoU (trained on the full Part-A **training split**). State the exact image counts of each so the label-efficiency ratio is explicit (e.g. "fine-tuned on N_{val} labelled images vs. N_{train} for the Part-A baseline").
- A short discussion of the mIoU gap: which SSL method closes the gap to full supervision best, and with how much less labelled data?

3.6. Task F — Error Analysis

- For each SSL method, visualize the worst-performing test images (lowest per-image IoU) against ground truth.
- Discuss whether errors concentrate on the **same** classes/images as your Part-A supervised models, or on **different** ones — and hypothesize why (e.g. SSL encoders may transfer texture/shape priors differently than a purely supervised backbone).

4. Required Notebook Structure

Every group submits a **data-prep notebook**, **one notebook block per SSL method** (either a single combined notebook, or a pretrain+downstream pair — see the epoch-timing note in Task B), and a **final comparison notebook**. Total notebook count is therefore **6 to 10**, depending on how many of the four methods your group needs to split.

Block	Contents
Data Prep (1 notebook)	Task A — load Part-A train/validation/test splits and map their SSL roles (train → unlabelled, validation → labelled fine-tune, test → monitoring/eval), per-method augmentation pipelines, sanity visualization.
SimCLR (1 or 2 notebooks)	**If combined:** Task B (SimCLR, 50 epochs) → Task C → Task D (50 epochs) → Task E → Task F, in one notebook. **If split:** a *pretraining* notebook (Task B, 50 epochs, saves the encoder checkpoint) and a *downstream* notebook (loads that checkpoint → Task C → Task D, 50 epochs → Task E → Task F).
BYOL (1 or 2 notebooks)	Same choice as SimCLR, using BYOL.
MAE (1 or 2 notebooks)	Same choice as SimCLR, using MAE.
DINOv2 (1 or 2 notebooks)	Same choice as SimCLR, using DINOv2.
Final Comparison (1 notebook)	Pulls saved metrics from every method's downstream stage **and** the Part-A `results.json`; produces the label-efficiency comparison table/chart (Task E) and consolidated error analysis (Task F).

Naming — Keep It Self-Explanatory, Not Fixed

There is no mandatory notebook filename. Name each notebook so a reader can immediately tell the method and stage it covers (e.g. `<dataset>-simclr` for a combined notebook, or `<dataset>-mae-pretrain` / `<dataset>-mae-downstream` for a split pair). List every notebook's title and link explicitly in your Submission post (Section 5) and in the report (Section 6.4), so the mapping from method → notebook(s) is unambiguous.

Every combined notebook, and every pretrain/downstream pair, must together cover this internal order: (1) setup & imports with version pins, (2) load the shared train (unlabelled), validation (labelled fine-tune), and test (monitoring/eval) splits from the data-prep notebook, (3) SSL pretraining stage on the train split — 50 epochs, pretext-loss curve, per-epoch time reported (and, if split, the encoder checkpoint saved as a Kaggle output/dataset here), (4) decoder attachment & frozen/fine-tuned configuration (loading the checkpoint first, if split), (5) fine-tuning config cell and training loop on the validation split — 50 epochs, with per-epoch monitoring loss/mIoU on the test split, per-epoch time reported, (6) final test-set evaluation (Task E metrics), (7) error-analysis visualizations (Task F), (8) markdown summary cell that appends results to the shared `results.json` used by the final comparison notebook.

5. Submission Instructions

Two Section Deadlines — Read Carefully

This course runs across two sections with **different** submission deadlines. Submit to Google Classroom according to **your own section**; a submission after your section's deadline is subject to the course late-policy unless otherwise announced.

Section	Deadline
Section 3	17.08.2026, 11:59 PM
Section 4	18.08.2026, 11:59 PM

1. Run each notebook **end-to-end with all cell outputs visible** before submission.

2. Set each Kaggle notebook to **Public**.

3. Compile your final report (Section 6) and export it as a PDF.

4. Submit **every** Kaggle link (6 to 10, depending on which methods you split — see Section 4) **and** the report PDF in a single post to **Google Classroom**, listing one line per notebook so the method → notebook(s) mapping is clear:

```
Group:  <group number/name>
Section:  <3 or 4>
Dataset:  <dataset name -- same as Part A>
Data Prep:  <kaggle link>
SimCLR (combined or pretrain + downstream):  <kaggle link(s)>
BYOL (combined or pretrain + downstream):  <kaggle link(s)>
MAE (combined or pretrain + downstream):  <kaggle link(s)>
DINOv2 (combined or pretrain + downstream):  <kaggle link(s)>
Final Comparison:  <kaggle link>
Report PDF: <attached / drive link>
```

6. Final Report — Structure & Format

Report Format Requirements

- **LaTeX, two-column layout** is mandatory. Any standard two-column conference-style template is acceptable (e.g. IEEE conference template, ACM sigconf, or your own two-column class) — the visual template is your group's choice, the section list below is not.
- **No Literature Review / Related Work section is required** for this assignment.
- An **Insights** section (Section 6.9 below) is mandatory and will be checked specifically for original analysis, not restated results.
- Typical length: 4–6 pages including figures and references, single-spaced, standard two-column conference margins.

6.1. 6.1 — Title, Authors & Abstract

Group name/number, all member names & IDs, dataset name. Abstract: 150–200 words summarizing approach and headline label-efficiency finding.

6.2. 6.2 — Introduction

Problem statement (semantic segmentation), motivation for self-supervised pretraining under limited labels, and a one-paragraph summary of what Part A established (best supervised architecture) that Part B builds on.

6.3. 6.3 — Dataset & Split

Dataset description (reused from Part A), the leakage-safe train/validation/test split, how each split's role was mapped for Part B (train → unlabelled pretraining, validation → labelled fine-tuning, test → downstream monitoring *and* final evaluation), exact image counts per split, and an explicit note on the test split's double role (see Section 2's caveat box).

6.4. 6.4 — Methodology

For each of the four SSL methods: pretext task description, backbone used, pretraining configuration (checkpoint-continuation vs. from-scratch, epochs, augmentations), and how the Part-A decoder head was attached (including any resolution-adapter needed for ViT-based encoders).

6.5. 6.5 — Fine-tuning Setup

Frozen-vs-fine-tuned decision and justification, fine-tuning hyperparameters (per method, in a single consolidated table), and training curves.

6.6. 6.6 — Results

Test-set metrics (mIoU, per-class IoU, pixel accuracy, Dice) for all four SSL methods, presented alongside the Part-A fully-supervised baseline in one comparison table and one chart.

6.7. 6.7 — Error Analysis

Qualitative failure cases per method, confusion patterns, and comparison against Part-A failure modes.

6.8. 6.8 — Label-Efficiency Discussion

Direct discussion of the central question: how much of the Part-A (full training-split label) performance is recovered by each SSL method using only the much smaller validation-split labels? State the exact label-count ratio for your dataset and rank the four methods by label-efficiency.

6.9. 6.9 — Insights (Mandatory)

What Goes Here

This is **not** a restatement of the Results table. Write your own analysis of, e.g.: why a particular pretext task (contrastive vs. masked-reconstruction vs. self-distillation) suited your dataset's visual structure better or worse; whether encoder freezing helped or hurt for a small labelled set; what you would try next with a larger compute budget; and any surprising or counter-intuitive result and your best explanation for it.

6.10. 6.10 — Conclusion & References

Brief conclusion (3–5 sentences) and a references list (SimCLR, BYOL, MAE, DINOv2 papers plus any library documentation cited).

7. AI Usage Policy

AI Tools — Allowed, but Understand What You Submit

- You **may** use AI coding assistants (including Claude, ChatGPT, GitHub Copilot, etc.) for boilerplate code, debugging error messages, and understanding library APIs.
- You **may not** have an AI tool generate your Insights section, your error-analysis reasoning, or your viva answers. These must reflect your own understanding — the viva will test this directly.
- **If you get stuck** — on a bug, a confusing API, an unexpected loss curve, or anything else — your **first stop should be the instructor or the graduate teaching assistant (TA)**, not an AI tool. Office hours and the course Q&A channel exist precisely so issues get resolved with someone who can also check your understanding, not just your code.
- If you do use an AI tool for a non-trivial code block, add a one-line comment in the notebook noting that (e.g. `# debugged with AI assistance`). This is not penalized — undisclosed, unexplainable AI-generated blocks are what gets penalized at viva.
- Any group member unable to explain a line of their own notebook — AI-assisted or not — will have their individual viva mark reduced regardless of the notebook's output quality.

8. Approved Checkpoints & Libraries

Reference Only — Full Starter Code Provided Separately

Example/starter notebooks for all four methods will be posted on Google Classroom. The table below is for choosing checkpoint sizes appropriate to your dataset and Kaggle GPU memory (∼16 GB, T4/P100); report your exact choice in the config cell.

Method	Common Library / Source	Notes
SimCLR	`lightly`, `solo-learn`, or official Google Research SimCLR repo	ResNet-50 backbone; NT-Xent temperature is a key hyperparameter to report.
BYOL	`lightly`, `byol-pytorch`	Needs online + target (EMA) network pair; report EMA decay rate.
MAE	Official Facebook Research `mae` repo, or HuggingFace `ViTMAEModel`	Report masking ratio (default 75%) and patch size.
DINOv2	`torch.hub` (`facebookresearch/dinov2`) or HuggingFace `Dinov2Model`	ViT-S/14 recommended for Kaggle GPU memory; report patch size and number of crops.

9. Viva Voce — Format and Expectations

Viva Focus — Pretext Tasks, Adapters, and Label-Efficiency Reasoning

Every group member must attend and must be able to independently explain any cell in any of the 6 notebooks and any claim in the report, especially in the Insights section. Date and schedule will be announced on Google Classroom after the submission deadline for your section.

9.1. Expected Coding Questions

- Walk through your split-role mapping: how did you load the Part-A train/validation/test file lists unmodified, and how did you verify no re-shuffling or leakage was introduced?
- The test split is used both to monitor fine-tuning and to report final metrics. Explain why this is a departure from strict practice and what it does to your reported numbers.
- For SimCLR/BYOL: show your augmentation pipeline for generating the two views. Which transforms are essential to the pretext task and why?
- For MAE: show your patch-masking code. What fraction of patches are masked, and what happens if you mask too few or too many?
- For DINOv2: explain your multi-crop augmentation (global vs. local crops) and how the student/teacher loss is computed.
- Show the adapter code that reshapes your ViT-based encoder's patch tokens back into a 2-D feature map before your Part-A decoder. Why is this reshaping necessary?
- Walk through your frozen-vs-fine-tuned experiment: what changed in the optimizer setup, and what did you observe?

9.2. Expected Theory Questions

- What does "self-supervised" mean, precisely — how does the pretext task supervise the network without human class labels?
- Explain the NT-Xent (normalized temperature-scaled cross-entropy) loss in SimCLR. What role do negative pairs play?
- Why does BYOL not collapse to a trivial constant representation despite having no negative pairs? What role does the stop-gradient / EMA target network play?
- In MAE, why is a very high masking ratio (e.g. 75%) both feasible and beneficial, unlike in NLP masked-language-modeling (typically ∼15%)?

- What is knowledge self-distillation in DINOv2? How do the student and teacher networks differ, and how is the teacher updated?
- What is "linear probing" vs. "fine-tuning" an SSL encoder? What does a large gap between the two suggest about the pretrained representation's quality?
- Why can an SSL-pretrained encoder often match a fully-supervised encoder's downstream accuracy using far fewer labels? What is being transferred?
- What is representation collapse, and which of the four methods here are architecturally designed to prevent it, and how?

10. Suggested Marking Scheme

(editable — confirm final weights before distributing to students)

Component	Marks
Data Prep — split role mapping & sanity checks (Task A)	8
SSL Pretraining — 4 methods (Task B)	24
Decoder Attachment & Adapter (Task C)	8
Supervised Fine-tuning — 4 methods (Task D)	20
Testing, Metrics & Label-Efficiency Comparison (Task E)	15
Error Analysis (Task F)	10
Report — structure, Insights section, formatting (Section 6)	15
Submission Subtotal	100

Viva Voce: weighted separately per course policy. Individual marks may differ within a group based on each member's ability to explain their own code and the reasoning in the Insights section.

11. Academic Integrity

- All code and report text must be your group's own work; see Section 7 (AI Usage Policy) for what AI assistance is and is not permitted.
- You may reference official documentation and papers (SimCLR, BYOL, MAE, DINOv2, library docs) and must cite them in markdown cells and in the report.
- Identical notebooks, identical reports, or near-identical numerical results across different groups will be investigated as an academic integrity concern.
- Inability to explain your own code or report claims at viva will reduce your individual viva mark regardless of output quality.

Part B — Final Submission Checklist

1. **Data Prep notebook**: Part-A train/val/test splits loaded unmodified → roles mapped (train=unlabelled, validation=labelled fine-tune, test=monitoring+eval) → per-method augmentation pipelines → sanity visualization
2. **SimCLR, BYOL, MAE, DINOv2** (1 notebook each, or a pretrain+downstream pair if per-epoch time >10 min): pretrained on train split (50 epochs), decoder-attached, fine-tuned on validation split (50 epochs, monitored on test split), tested, error-analyzed
3. **Final Comparison notebook**: consolidated label-efficiency comparison against the Part-A baseline

4. **Report**: two-column LaTeX PDF, no literature review, Insights section included
5. All notebooks (6–10, depending on splits) set to **Public** on Kaggle
6. All notebook links + report PDF submitted together in **Google Classroom**, matching **your section's** deadline (Sec. 3: 17.08.2026 / Sec. 4: 18.08.2026)
7. Viva to follow — be ready to explain any cell, any claim

References:

[1] Chen et al., *A Simple Framework for Contrastive Learning of Visual Representations (SimCLR)*. ICML 2020.

[2] Grill et al., *Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning (BYOL)*. NeurIPS 2020.

[3] He et al., *Masked Autoencoders Are Scalable Vision Learners (MAE)*. CVPR 2022.

[4] Oquab et al., *DINOv2: Learning Robust Visual Features without Supervision.* arXiv:2304.07193, 2023.

[5] Caron et al., *Emerging Properties in Self-Supervised Vision Transformers (DINO)*. ICCV 2021.

Note: This is the final part of the CSE 438 segmentation assignment sequence (Part A + Part B).